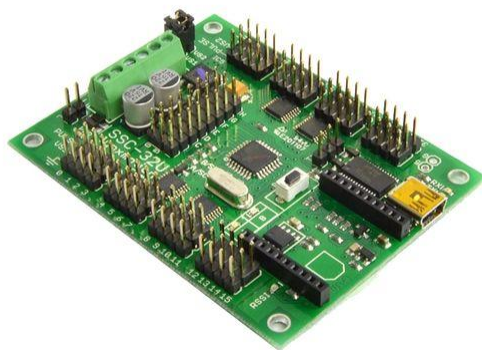




Carte contrôleur de servomoteurs Lynxmotion SSC-32U USB

Guide électronique — Traduction française



Révisions

V1.2 — Juillet 2017

Retours / Errata / Support : robotshop.com/forum/ssc-32u-f142

Table des matières

Table des matières	2
Vue d'ensemble	4
Servomoteurs : numériques, analogiques et « intelligents »	5
Servomoteurs analogiques	5
Servo à rotation continue	6
Servo numérique R/C	6
Servo « intelligent » (Smart Servo)	6
Informations matérielles	8
Dimensions	8
Broches S1	8
Broches S2	9
Broches d'E/S analogiques	9
Borniers à vis	10
VS = VL	11
VS1 = VS2	12
XBee	12
USB	13
LED Tx / Rx	13
Liaison série (Serial)	14
LED A / B	14
Broches 5G	15
RSSI	15
PWR	16
Options d'alimentation	17
USB	17
Bornier VL	17
Bornier VS1	17
Bornier VS2	17
Cavaliers VS1 = VS2	17
Cavalier VL = VS	18
Considérations sur l'alimentation	19
Adaptateur secteur	19
Chimie des batteries et courant	19

Options de communication / contrôle.....	21
USB.....	21
UART TTL (broches TX / RX / G)	22
Connecteurs XBee.....	23
SSC-32U / BotBoarduino / PS2.....	23
Commandes	25
Types et groupes de commandes	25
Séquenceur hexapode 12 servos	28
Configuration de la démarche	30
Fonctions avancées.....	32
Annuler une sortie (Cancel Output).....	32
Sortie tout-ou-rien (Discrete Output)	32
Sortie par octet (Byte Output)	32
Interroger l'état du mouvement — Q <cr>.....	32
Interroger la largeur d'impulsion — QP <arg> <cr>.....	33
Entrée numérique (Digital Input).....	33
Entrées analogiques (Analog Inputs)	33
Débit (Baud)	33
Firmware et registres.....	35
Registres de la SSC-32 — informations générales	35
Registre Enable (R0) — définition des bits	35
Lecture / écriture de registre.....	36
Commandes de registre diverses.....	36
Chaînes de démarrage (Startup Strings)	36

Vue d'ensemble

La carte **Lynxmotion SSC-32U** est un contrôleur de servomoteurs R/C polyvalent et facile à utiliser, dont le cœur est un microcontrôleur **Atmel ATmega328p**. Ses caractéristiques principales sont :

- Entrée par USB, série ou XBee
- Contrôle jusqu'à 32 servomoteurs
- Séquenceur intégré pour hexapode à 12 degrés de liberté (12 DoF)
- 8 broches d'entrée analogiques
- Protocole de commande simple à comprendre
- Fonctions avancées
- Idéale pour les bras robotisés ou les robots à pattes
- Compatible avec le logiciel graphique FlowBotics Studio

⚠ La SSC-32U est un contrôleur de servomoteurs dédié : la carte n'est pas destinée à être « programmée » (elle ne stocke pas de code créé par l'utilisateur). Elle se contente de recevoir et d'exécuter les commandes envoyées par un appareil externe (ordinateur ou microcontrôleur). Utiliser un contrôleur dédié libère la mémoire du microcontrôleur, qui n'a plus à mettre à jour en permanence la position des servos.

La SSC-32U est largement basée sur la SSC-32 (sa devancière).

Servomoteurs : numériques, analogiques et « intelligents »

Servomoteurs analogiques

Notions de base sur les servos R/C

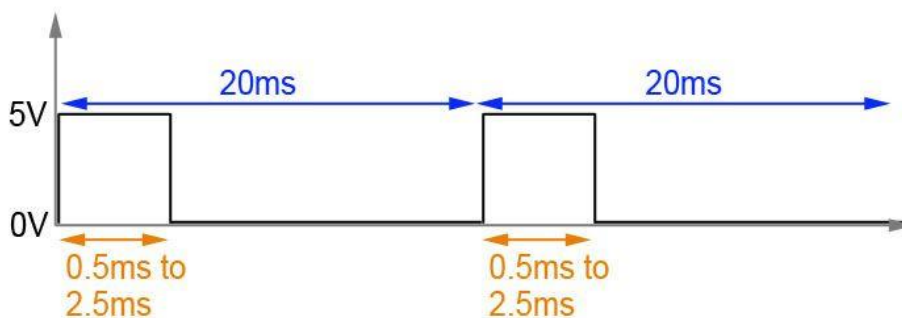
Pour bien comprendre un servomoteur télécommandé (« R/C »), il faut connaître la terminologie suivante :

- Boîtier (généralement en plastique, parfois en aluminium)
- Moteur à courant continu (généralement à balais, parfois sans balais ou plus exotique)
- Engrenages (qui réduisent la vitesse du moteur et augmentent le couple)
- Cannelure / arbre (liaison du dernier engrenage interne ; s'emboîte dans le palonnier pour le faire tourner)
- Palonnier (la pièce en plastique ou métal qui permet de fixer d'autres éléments)
- Potentiomètre (capteur de position absolue qui renseigne la position du servo)
- Électronique (qui contrôle le mouvement)
- Câble et connecteur à 3 broches (espacement standard 0,1 pouce)



Connecteur 3 broches : Blanc/jaune/orange = Signal (impulsion) ; Rouge = 4,8 V à 6 V (VS) ; Noir/marron = Masse (GND)

Les servos R/C utilisent une forme de **modulation de largeur d'impulsion (PWM)**. L'électronique du servo attend un signal 5 V répété, et mesure la durée pendant laquelle ce signal reste actif. Pour la plupart des servos R/C, une impulsion de 500 μ s (0,5 ms) à 5 V fait tourner le palonnier à -90° , une impulsion de 2500 μ s (2,5 ms) le fait tourner à $+90^\circ$, et une impulsion de 1500 μ s correspond donc à 0° (position centrée). Cette impulsion doit être répétée toutes les ~ 20 ms.



Impulsion temporisée R/C (répétée toutes les 20 ms)



Servo standard Hitec : angle de fonctionnement en fonction du signal

Lorsqu'un servo R/C atteint un angle donné, il s'efforce de conserver cette position tant qu'il reçoit le signal toutes les 20 ms. Si le couple demandé dépasse ce que le servo peut fournir, le courant augmente fortement, ce qui fait chauffer le servo et finit par le détruire.

La plupart des servos R/C ont un **rapport cyclique d'environ 25 %**. Ce rapport cyclique indique la proportion de temps où le moteur doit être actif par rapport au temps de repos. Cela ne signifie pas qu'on peut le faire fonctionner 1 heure puis le laisser refroidir 3 heures : il est conseillé de ne pas dépasser environ 10 minutes de fonctionnement continu, après quoi le servo a besoin d'environ 30 minutes de refroidissement.

Servo à rotation continue

Un servo R/C à rotation continue utilise les mêmes impulsions / le même contrôle qu'un servo analogique classique, mais convertit les impulsions en vitesse et en direction : 1,5 ms correspond à l'arrêt, 0,5 ms à la vitesse maximale en sens antihoraire, et 2,5 ms à la vitesse maximale en sens horaire. Les valeurs intermédiaires donnent des vitesses intermédiaires. Un servo à rotation continue ne peut pas être placé à un angle précis (ou relatif) : il fonctionne comme un motoréducteur à courant continu.

Servo numérique R/C

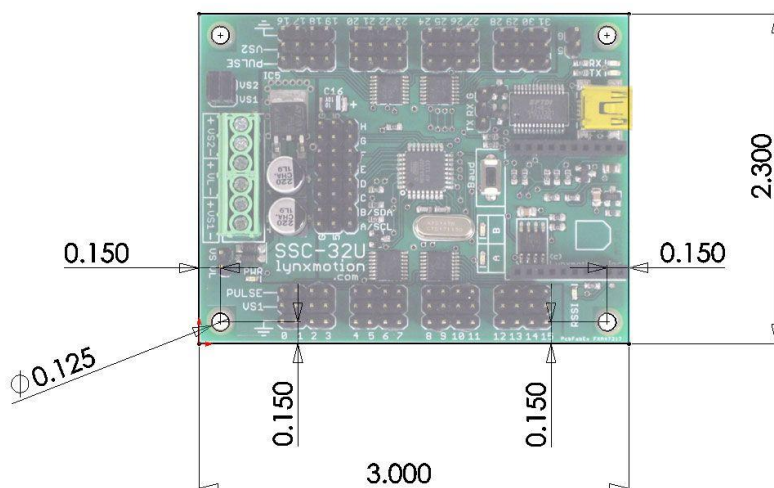
Un servo numérique fonctionne de la même manière qu'un servo analogique R/C classique, mais autorise souvent des mouvements plus précis et plus rapides. Dans bien des cas, les limites, la position centrale (et divers autres paramètres) peuvent être modifiés à l'aide d'un programmeur de servo (la SSC-32U n'est pas un programmeur de servo numérique). Une fois programmé, le servo s'utilise comme n'importe quel servo analogique R/C. Les servos numériques ressemblent extérieurement aux servos analogiques.

Servo « intelligent » (Smart Servo)

Un « servo intelligent » n'est normalement pas piloté par un signal R/C, mais par des commandes série. Son connecteur diffère généralement de celui d'un servo R/C. La SSC-32U n'est pas conçue pour piloter des servos intelligents. Ces derniers ont souvent une conception mécanique différente ou personnalisée par rapport aux servos R/C.

Informations matérielles

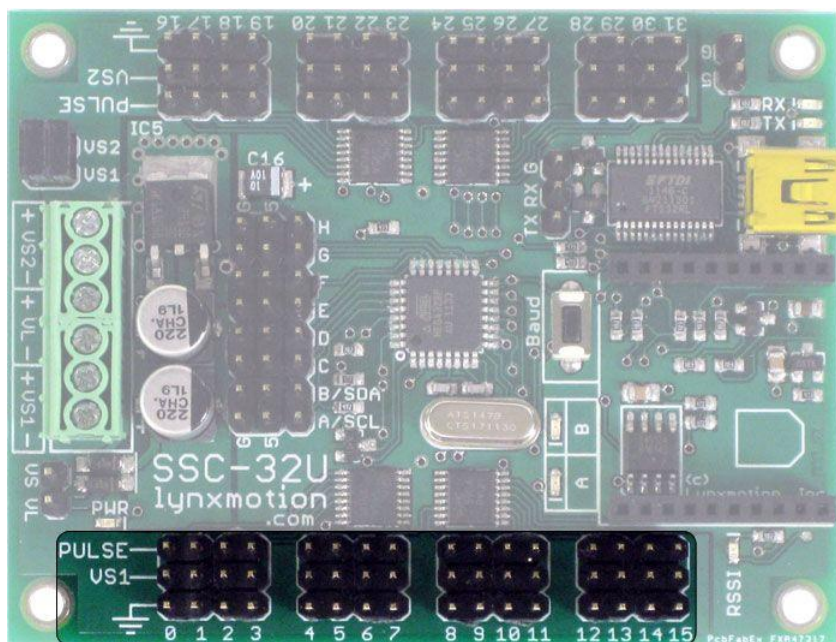
Dimensions



Dimensions de la carte SSC-32U (en pouces)

La SSC-32U mesure 3,00" × 2,30", avec des trous de 0,125" situés à 0,15" de chaque bord. Elle a été conçue aux mêmes dimensions que la Lynxmotion BotBoarduino, la SSC-32 (sa devancière), la Bot Board et la Bot Board 2, afin de pouvoir être empilée avec elles.

Broches S1

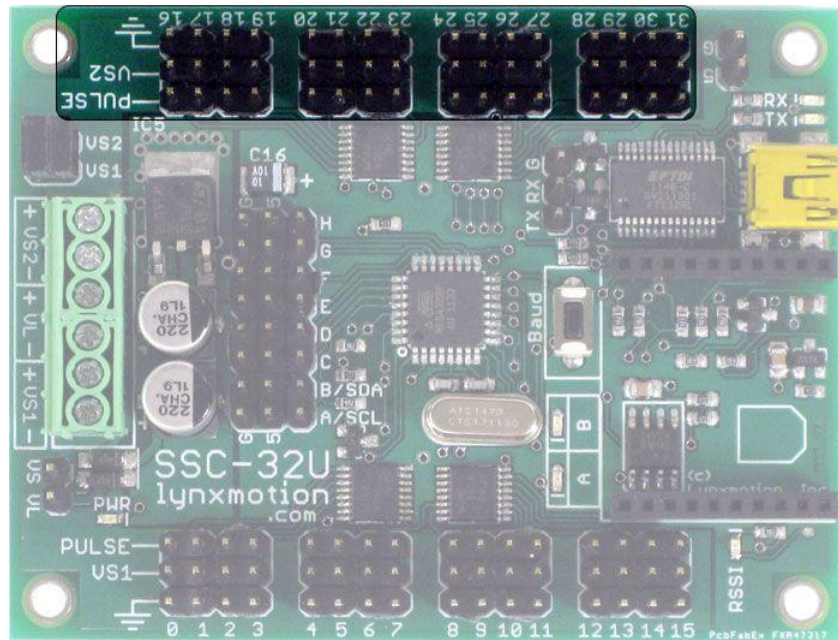


Section S1 (broches 0 à 15)

La SSC-32U peut contrôler jusqu'à 32 servos, répartis en deux sections distinctes de 16 servos, regroupés par jeux de quatre. Les broches 0 à 15 correspondent à VS1. La broche la plus externe (la plus éloignée

du centre de la carte) est la masse (normalement le fil noir d'un câble 3 broches). La broche centrale correspond à la tension (normalement le fil rouge) et est repérée VS1. La dernière broche est la broche de signal, qui envoie la commande de position au servo. La couleur du fil de signal varie, mais elle est généralement blanche ou jaune.

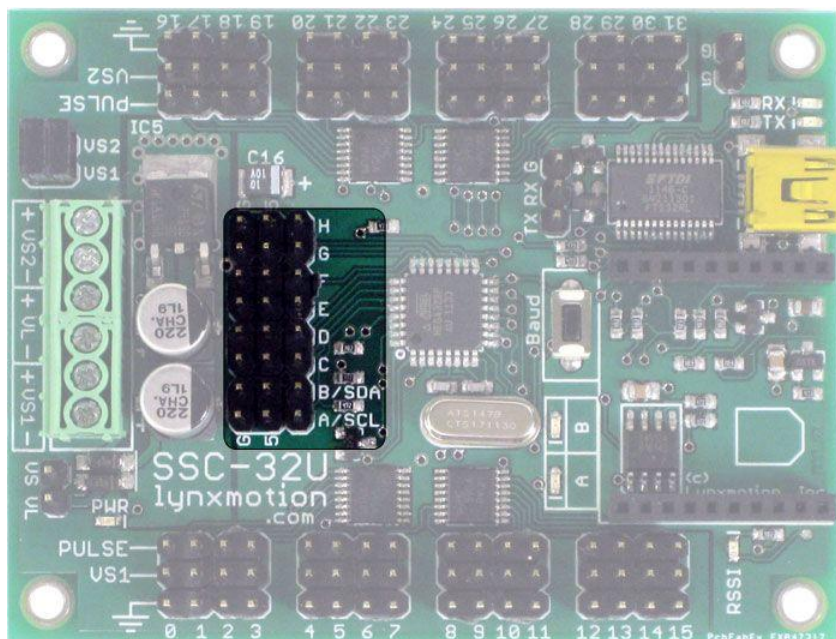
Broches S2



Section S2 (broches 16 à 31)

Il s'agit du second jeu de 16 broches. Les broches 16 à 31 correspondent à VS2. La broche la plus externe est la masse (fil noir), la broche centrale correspond à la tension (fil rouge) et la dernière broche est la broche de signal qui envoie la commande de position au servo. La couleur du fil de signal est généralement blanche ou jaune.

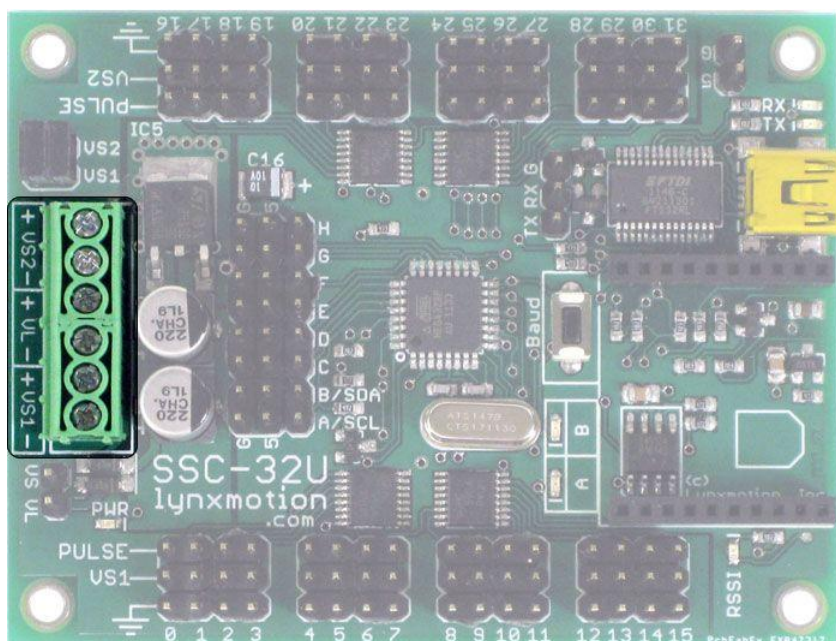
Broches d'E/S analogiques



Broches d'entrée/sortie analogiques (A à H)

La carte comporte 8 broches d'E/S repérées de « A » à « H ». La rangée de broches de gauche, marquée « G », correspond à la masse (fil noir). La rangée centrale, marquée « 5 », fournit une sortie 5 V (fil rouge). La dernière rangée (sans repère) est destinée au signal. Deux de ces broches peuvent aussi servir à l'I2C : la broche A correspond à SCL et la broche B à SDA. Pour une communication I2C, il faut également relier la broche GND ; et si la SSC-32 alimente les autres périphériques I2C, il faut aussi relier l'une des broches 5 V.

Borniers à vis

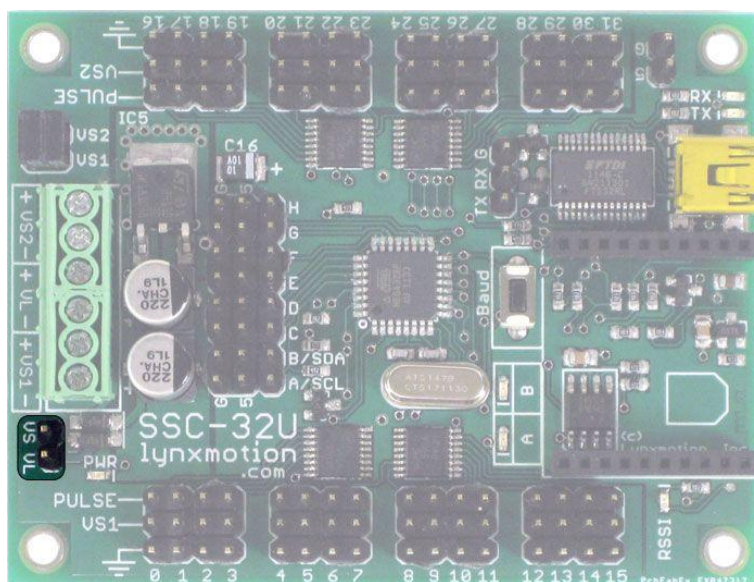


Borniers à vis (VS1, VS2, VL)

- VS1 alimente directement les broches 0 à 15 (idéalement 4,8 V à 6 V pour des servos R/C standard)
- VS2 alimente directement les broches 16 à 31 (idéalement 4,8 V à 6 V pour des servos R/C standard)
- VL alimente le contrôleur logique

Veillez à relier le fil positif (généralement rouge) au « + » et le fil négatif (généralement noir) au « - ». Assurez-vous qu'aucun fil dénudé ne touche les deux en même temps (court-circuit).

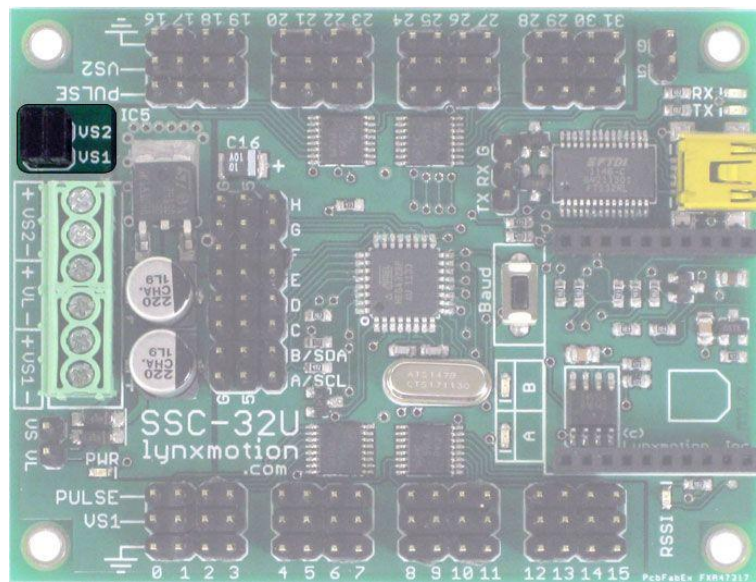
VS = VL



Cavalier VS = VL

Ces deux broches permettent de relier le bornier VS1 à VL (c.-à-d. VL = VS). Cela revient au même que de relier par un fil la borne VS1+ à la borne VL+. Un cavalier pour ces broches n'est pas fourni.

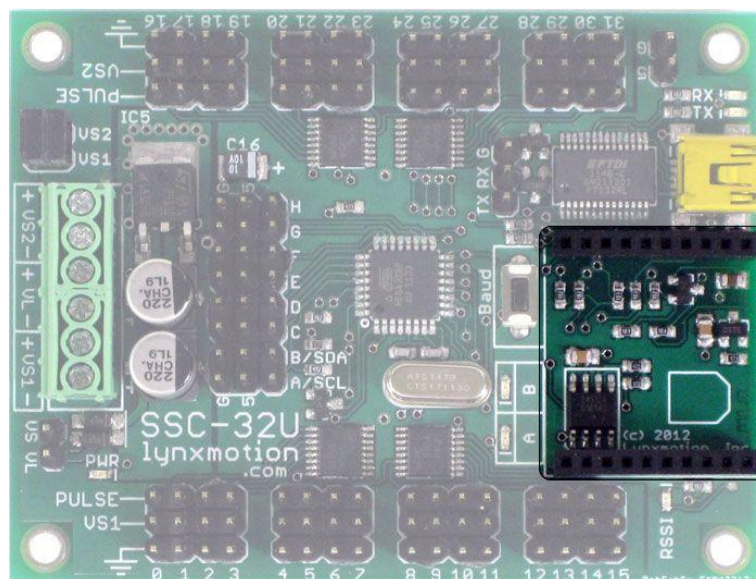
VS1 = VS2



Cavaliers VS1 = VS2

Ces broches 2x2 permettent de poser VS1 = VS2 (équivalent à relier la borne VS1+ à la borne VS2+ par un fil). Il y a deux cavaliers (et non un seul) en raison du courant en jeu. La carte est livrée avec ces cavaliers en place ; c'est à l'utilisateur de les retirer si nécessaire.

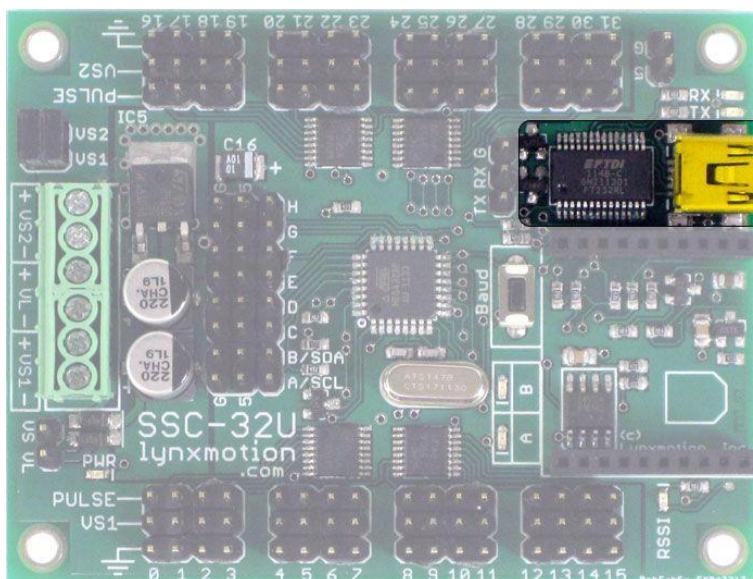
XBee



Connecteur XBee

Les connecteurs XBee permettent de raccorder un module sans fil XBee ou tout autre appareil compatible XBee. Respectez l'orientation indiquée par le sérigraphie de la carte.

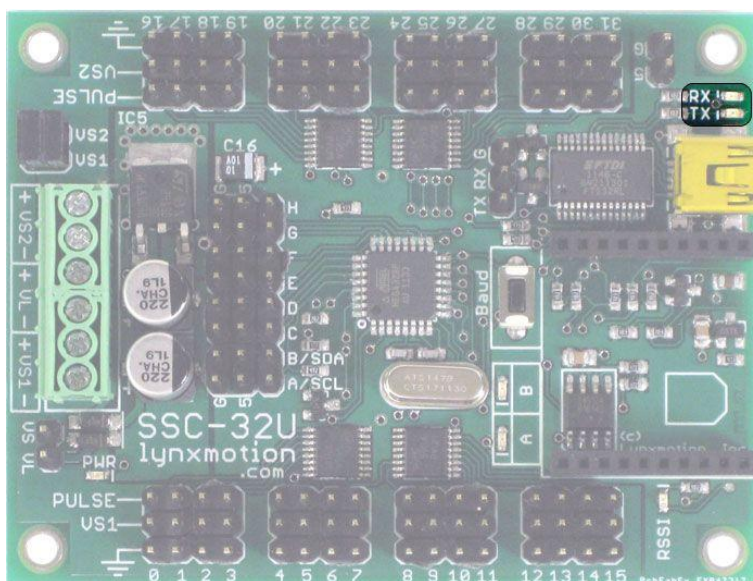
USB



Port USB

Les commandes peuvent être envoyées au microprocesseur de la SSC-32U depuis l'ordinateur via le port USB embarqué. Le microprocesseur ne comprend que les commandes au format série (et non USB) ; une puce FTDI (la zone située près du port USB) convertit donc le flux USB en série et permet à la carte d'être détectée comme un port COM une fois connectée.

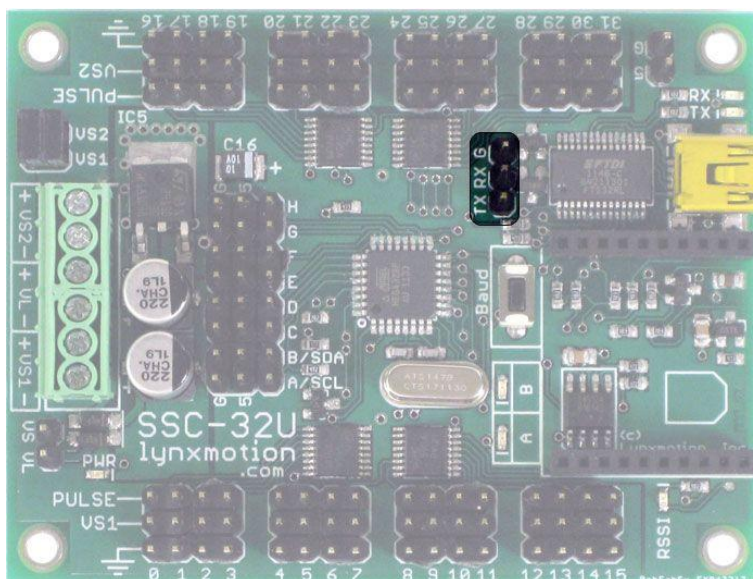
LED Tx / Rx



LED Tx / Rx

Les deux LED situées près du port USB donnent un retour visuel : réception de données (Rx) et émission de données (Tx). Elles ne concernent que la communication via le port USB, et non le connecteur XBee ni les broches série.

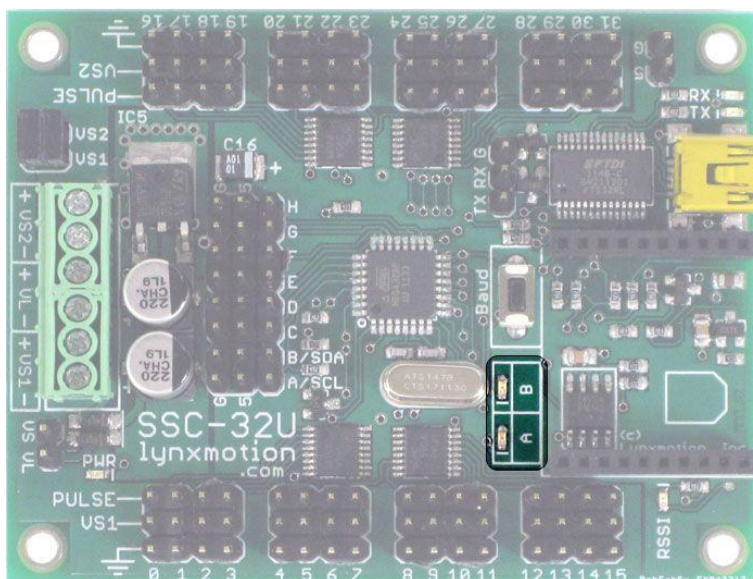
Liaison série (Serial)



Broches série (derrière la puce FTDI)

Les trois broches situées derrière la puce FTDI sont Tx, Rx et GND. Les données émises par la SSC-32U passent par la broche Tx, tandis que celles qu'elle reçoit passent par la broche Rx. Ces broches facilitent l'envoi de commandes depuis un autre microcontrôleur. Pour cela, reliez la broche Tx du microcontrôleur à la broche Rx de la SSC-32U, la broche Rx du microcontrôleur à la broche Tx de la SSC-32U, et GND à GND.

LED A / B



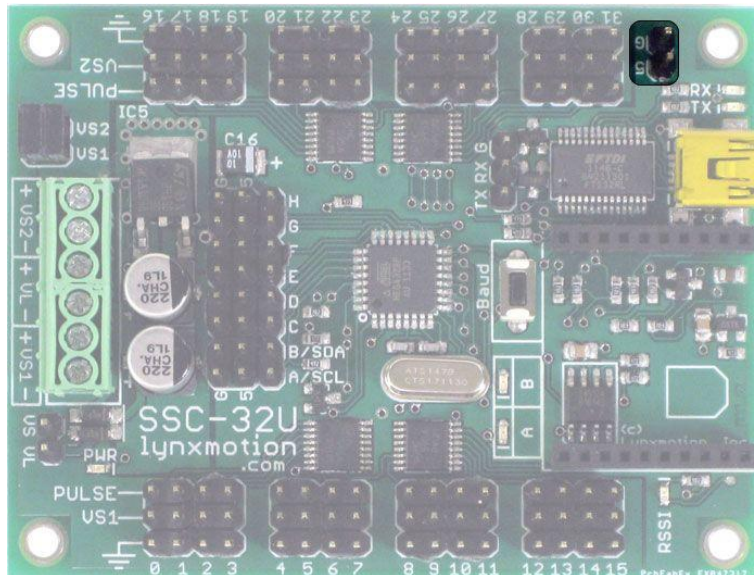
LED A et B

La carte comporte deux LED distinctes, repérées A et B. Hors réglage du débit (Baud), elles indiquent :

- Les deux allumées à la mise sous tension, avant réception du premier octet.

- Clignotement vert lorsqu'un octet valide est reçu.
- Clignotement rouge lorsqu'un octet invalide est reçu (erreur de trame).

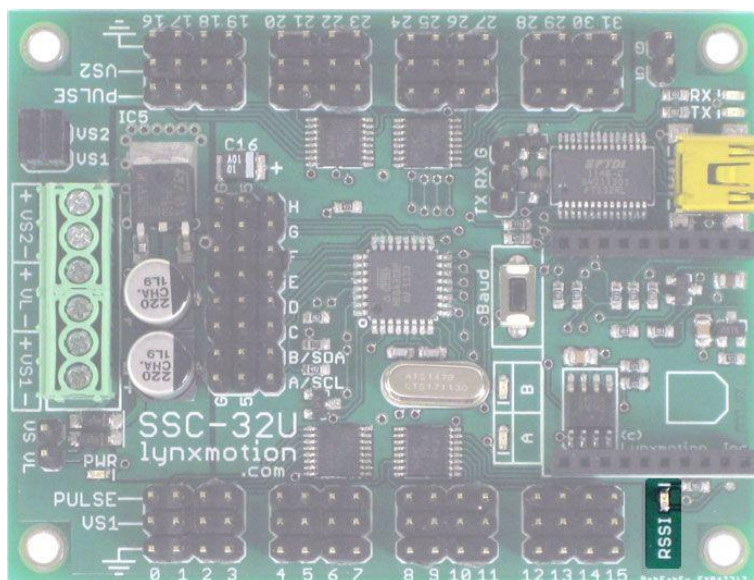
Broches 5G



Broches 5G

Ces broches ne sont normalement pas nécessaires et ne sont pas fournies. Si les deux condensateurs embarqués s'avéraient insuffisants pour votre application, un troisième condensateur peut être ajouté ici. Si vous n'êtes pas à l'aise en électronique, il est conseillé de ne pas toucher à cette partie de la carte.

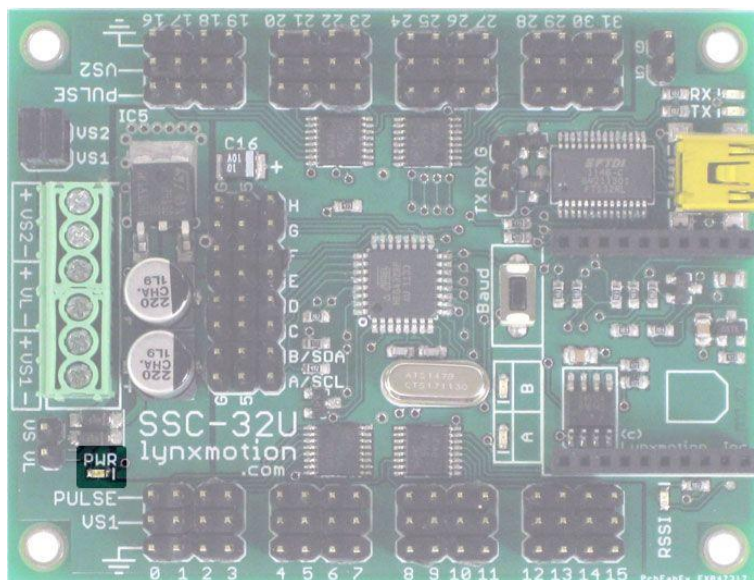
RSSI



LED RSSI

RSSI signifie « Received Signal Strength Indication » (indication de la force du signal reçu). Cette LED est reliée à la broche 6 du connecteur XBee. Différents intervalles de clignotement indiquent différents états, selon le module choisi.

PWR



LED PWR (alimentation)

La LED PWR est reliée à V_{logic}, qui correspond à MAX(V_L, V_{S1}) moins environ 0,7 V. C'est la tension ensuite régulée à 5 V pour alimenter les circuits logiques. La LED s'allume lorsque le microcontrôleur embarqué reçoit une alimentation suffisante.

Options d'alimentation

USB

Lorsque la carte est connectée à un ordinateur via USB, la puce USB-série embarquée est alimentée et l'ordinateur peut la détecter et installer les pilotes. En revanche, l'USB n'alimente pas la puce ATmega principale : celle-ci doit être alimentée séparément via VS1 ou VL.

Bornier VL

Les borniers VL acceptent des entrées non régulées pour la tension logique (la tension utilisée par le processeur principal). La tension logique est sélectionnée automatiquement entre VL et VS1 (la plus élevée des deux). Tant que VS1 est supérieure à 5,3 V (hors chutes de tension temporaires), elle suffit à alimenter la logique et il n'est pas nécessaire de relier quoi que ce soit aux borniers VL.

- Idéal : rien de connecté
- Nominal : 6 à 12 V
- Absolu : 5,3 à 16 V

Dans la plupart des cas, une seule batterie suffit (reliée à VS1), mais il peut être préférable d'avoir une batterie pour la logique et une autre pour les servos lorsque :

- Vous utilisez 4,8 V pour VS1, ce qui n'est pas suffisant pour VL ;
- Vous voulez couper l'alimentation des moteurs tout en gardant la logique alimentée ;
- Vous voulez garantir que la logique reste alimentée même si la batterie des servos est épuisée.

Bornier VS1

Le bornier VS1 est relié directement aux lignes d'alimentation et de masse des broches servo 0 à 15. La tension appliquée à VS1 doit idéalement correspondre à la tension nominale des servos. Pour la plupart des servos R/C, c'est 6 V. Certains servos « haute tension » peuvent fonctionner avec une batterie LiPo 7,4 V ou 11,1 V, mais sauf indication explicite dans les spécifications du servo, soyez prudent. Utiliser une LiPo 7,4 V avec un servo R/C standard est déconseillé, et 11,1 V détruira probablement un servo standard.

- Nominal : 6 V (servos R/C standard)
- Absolu : 0 à 16 V

Bornier VS2

Le bornier VS2 est relié directement aux lignes d'alimentation et de masse des broches servo 16 à 31. La tension appliquée à VS2 doit idéalement correspondre à la tension nominale des servos (6 V pour la plupart des servos R/C). Comme pour VS1, soyez prudent avec les servos « haute tension ».

Cavaliers VS1 = VS2

La seconde moitié des broches servo (16 à 31) peut être alimentée par la même source que VS1 en laissant en place les deux cavaliers VS1 = VS2. Il y a deux cavaliers (plutôt qu'un) à cause du courant. Si vous souhaitez alimenter les servos de VS2 séparément de VS1, retirez ces deux cavaliers. Si les deux cavaliers VS1 = VS2 sont en place, vous pouvez alimenter soit VS1 soit VS2, mais pas les deux. Retirer ces cavaliers est utile lorsque :

- Une seconde batterie est nécessaire à cause d'un courant élevé ;
- Le second jeu de servos fonctionne à une tension différente ;
- Vous voulez utiliser deux packs d'alimentation distincts.

Cavalier VL = VS

Le cavalier VL = VS force la tension logique à utiliser l'alimentation reliée à VS1. Sauf problème inhabituel avec la sélection automatique d'alimentation, ce cavalier ne doit pas être utilisé. Il n'est ni installé ni fourni. N'utilisez pas les borniers VL si vous installez ce cavalier.

Considérations sur l'alimentation

La carte SSC-32U ne mesure pas la tension de la batterie et continue de fonctionner même si la batterie se décharge profondément. C'est à l'utilisateur de veiller à maintenir la tension du pack au-dessus d'un certain seuil, sous peine d'endommager irrémédiablement la batterie. Symptômes d'une batterie déchargée :

- Les servos sont plus lents que la normale ;
- Les servos ne maintiennent pas leur position sous charge.

Adaptateur secteur

Pour alimenter la carte par un adaptateur secteur plutôt que par des batteries, tenez compte des points suivants :

- Un adaptateur secteur fournit une alimentation illimitée, mais les servos ne doivent pas fonctionner en continu plus de ~30 minutes ;
- L'adaptateur doit fournir un courant suffisant (en ampères) pour tous les servos ;
- La polarité du connecteur jack n'est pas toujours évidente : reliez bien positif à positif et négatif à négatif / GND ;
- La SSC-32U n'ayant pas de connecteur jack, un adaptateur est probablement nécessaire. Un adaptateur jack 2,1 mm vers bornier à vis et une paire de fils (ou un faisceau adapté) sont recommandés.



Options d'entrée par connecteur jack (barrel)

Chimie des batteries et courant

Alcaline

Les piles alcalines se présentent le plus souvent en 9V, AAA, AA, C et D. Non rechargeables, elles font généralement 1,5 V. Quatre cellules de 1,5 V donnent un pack de 6 V. La capacité est rarement indiquée et le débit de courant (en pointe ou en continu) l'est rarement aussi. Si vous utilisez plusieurs servos et qu'ils manquent de puissance, vos piles en sont probablement la cause.

NiMh / NiCd

Les NiMh et NiCd se présentent en AA, AAA ou en packs. Rechargeables, elles sont actuellement les plus populaires pour les robots mobiles (hors humanoïdes) en raison de leur poids et de leur prix. Les NiMh sont préférables aux NiCd, qui perdent de la capacité si on les recharge partiellement (effet mémoire). Une cellule fournit généralement 1,2 V ; il faut donc 5 cellules pour un pack de 6 V. La capacité est presque toujours indiquée sur la cellule. Le taux de décharge est généralement de 1C ou 2C. Le « C » se rapporte à la capacité : pour une cellule de 2800 mAh, 1C = 2800 mA. Par exemple, une cellule NiMh 1,2 V 2000 mAh notée 1C peut débiter 2,0 A en continu ; cinq de ces cellules fournissent donc 6 V à 2 A en continu.

Lithium

Les batteries LiPo / LiFe se présentent en cellules plates ou en packs rectangulaires, généralement par multiples de 3,7 V. Plus légères que les NiMh, elles offrent des taux de décharge bien plus élevés (10C, 20C, voire plus). Malheureusement pour les servos R/C, 3,7 V est trop faible et 7,4 V est (souvent) trop élevé. Les LiPo sont donc recommandées uniquement pour les servos haute tension.

Plomb-acide

Les batteries au plomb sont généralement en 6 V, 12 V et 24 V (entre autres). C'est le type de batterie des véhicules à essence. Un pack 6 V au plomb fournit en général assez de courant pour alimenter de nombreux servos R/C, mais la batterie elle-même est assez lourde.

Options de communication / contrôle

Il existe trois façons de communiquer avec la carte SSC-32U : USB, UART TTL et connecteur XBee. Il est recommandé de n'en utiliser qu'une seule ; en cas d'usage simultané, l'USB est prioritaire.

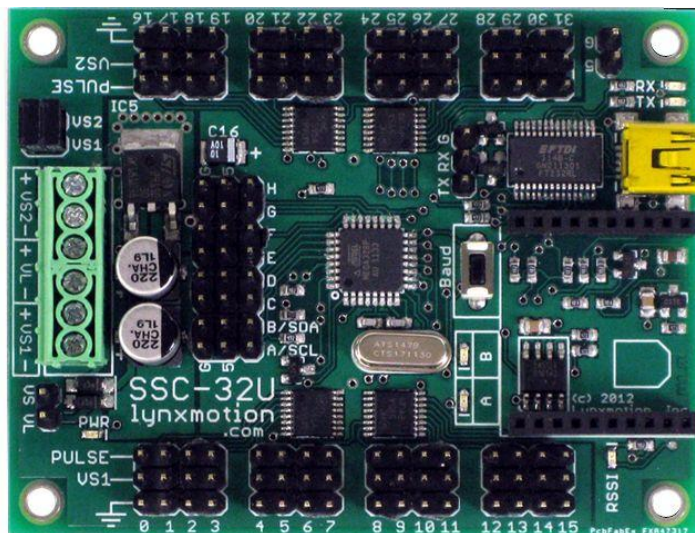


Photo de la SSC-32U

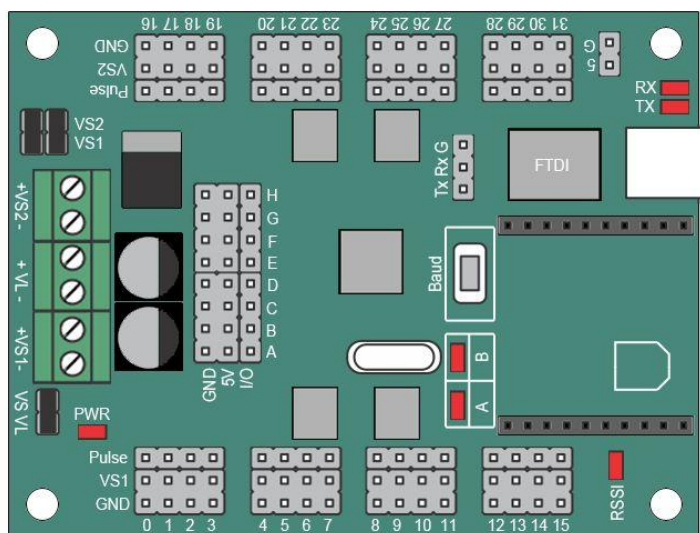
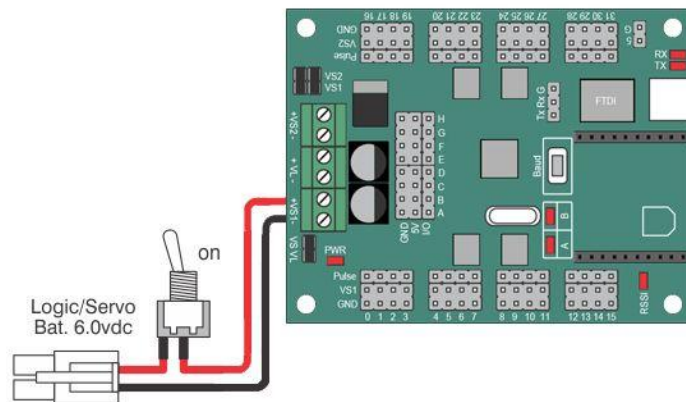


Schéma de la SSC-32U

USB

Vous pouvez relier la carte à un ordinateur de bureau ou portable avec un câble USB standard (mini-B vers A). Une fois le câble branché, l'ordinateur devrait détecter la carte et installer automatiquement les pilotes pour qu'elle apparaisse comme un port COM. S'il ne la détecte pas, téléchargez et installez manuellement les pilotes VCP de la puce FTDI selon votre système d'exploitation :

ftdichip.com/Drivers/VCP.htm



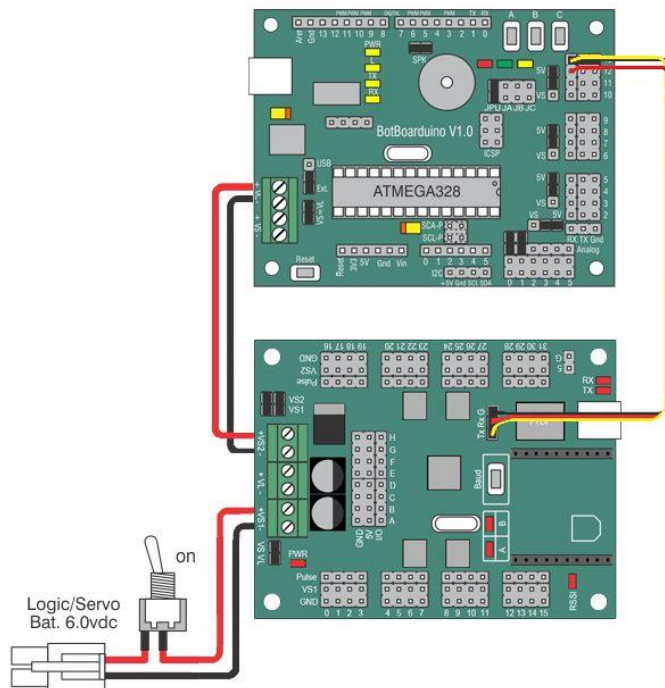
Alimentation de la SSC-32U

⚠ Tous les câbles USB ne se valent pas : certains ne transmettent que l'alimentation, sans la communication. En cas de problème de communication via USB, vérifiez d'abord l'alimentation, puis changez de câble USB.

UART TTL (broches TX / RX / G)

Les commandes peuvent être envoyées à la SSC-32 via les broches série de la carte. Il existe deux façons de relier la BotBoarduino à la SSC-32 :

1. Connexion série directe
 - TX de la BotBoarduino → RX de la SSC-32U
 - RX de la BotBoarduino → TX de la SSC-32U
 - GND de la BotBoarduino → GND de la SSC-32U
2. Série logicielle (Software Serial — permet de relier d'autres périphériques aux broches série de la BotBoarduino)
 - Broche d'E/S 13 de la BotBoarduino → TX de la SSC-32 (jaune)
 - Broche d'E/S 12 de la BotBoarduino → RX de la SSC-32 (rouge)
 - GND de la BotBoarduino → GND de la SSC-32 (noir)

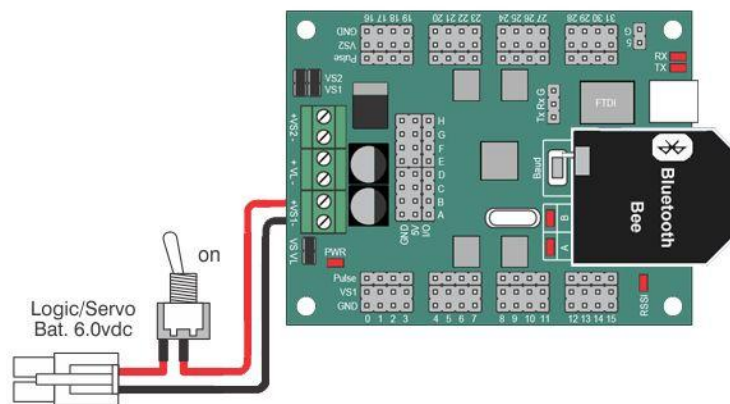


SSC-32U vers BotBoarduino / Arduino

Connecteurs XBee

Le connecteur XBee de la SSC-32U accepte divers périphériques sans fil, dont :

- Modules Bluetooth au format XBee
- Modules XBee
- Modules RF au format XBee

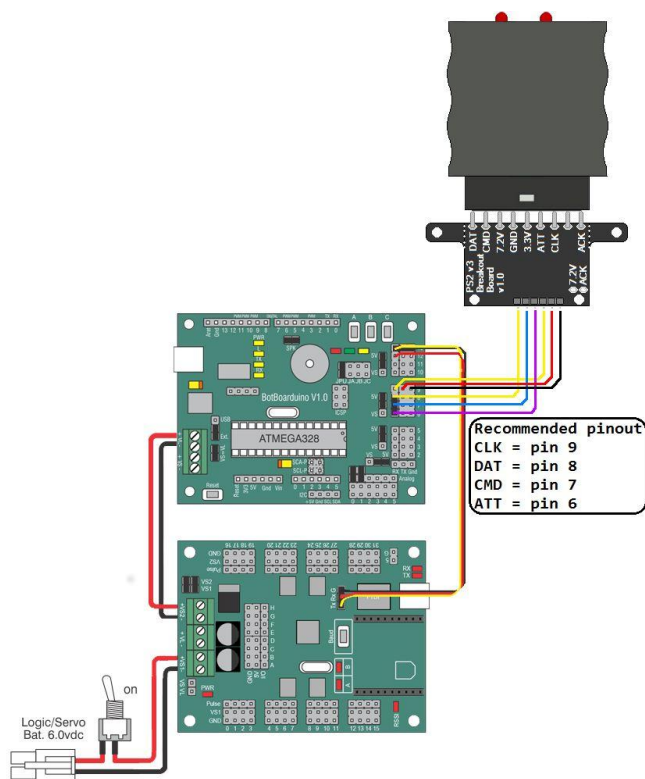


SSC-32U avec un module Bluetooth Bee installé

Notez que l'orientation du module doit correspondre à celle de la sérigraphie de la carte.

SSC-32U / BotBoarduino / PS2

La SSC-32U peut être utilisée avec la BotBoarduino (et les Arduino classiques), elle-même éventuellement reliée à une manette PS2. Le récepteur PS2 Lynxmotion et son adaptateur de niveau (level shifter) sont illustrés ci-dessous.



SSC-32U avec BotBoarduino et récepteur PS2 V3 Lynxmotion + adaptateur de niveau

Brochage recommandé : CLK = broche 9 · DAT = broche 8 · CMD = broche 7 · ATT = broche 6.

Commandes

Types et groupes de commandes

Commande pour un seul servo

Pour positionner un servo, la SSC-32 doit recevoir une commande série au format suivant (les signes « inférieur à » et « supérieur à » ne font pas partie de la commande ; les valeurs en italique sont facultatives) :

```
# <ch> P <pw> S <spd> T <time> <cr>
```

- **<ch>** : broche / canal auquel le servo est relié (0 à 31), en décimal
- **<pw>** : largeur d'impulsion désirée (normalement 500 à 2500), en microsecondes
- **<spd>** : vitesse de déplacement du servo, en microsecondes par seconde
- **<time>** : temps (en microsecondes) pour aller de la position actuelle à la position désirée. Affecte tous les servos (65535 max)
- **<cr>** : retour chariot (ASCII 13)

Exemple 1 :

```
#5P1500S750<cr>
```

Le servo relié à la broche n°5 se déplace en position 1500 (0° / centré) à une vitesse de 750 µs par seconde. Les arguments numériques de toutes les commandes SSC-32 doivent être des chaînes ASCII de nombres décimaux (ex. « 1234 »). Certaines commandes acceptent les nombres négatifs (ex. « -5678 »). Le format ASCII n'est pas sensible à la casse. Les espaces, tabulations et sauts de ligne sont ignorés.

⚠ Pour bien comprendre l'argument de vitesse : 1000 µs de course correspondent à environ 90° de rotation. Une vitesse de 100 µs/s signifie que le servo mettra 10 secondes (1000 ÷ 100) pour parcourir 90°. À l'inverse, 2000 µs/s équivaut à 500 ms (0,5 s) pour 90° (1000 ÷ 2000). Les servos ayant une vitesse maximale physique, une consigne plus rapide restera limitée.

Toutes les commandes SSC-32 doivent se terminer par un retour chariot (ASCII 13). Sous Arduino, on peut l'obtenir avec `Serial.println()`. Plusieurs commandes du même type peuvent être émises simultanément dans un « groupe de commandes » ; toutes les commandes du groupe sont exécutées après réception du retour chariot final.

Exemple 2 :

```
#3 P1600 T1000 <cr>
```

Cet exemple déplace le servo 3 en position 1600. Le déplacement prendra 1 seconde, quelle que soit la distance à parcourir.

Commande multi-servos (« groupe de commandes »)

La SSC-32U permet de recevoir plusieurs commandes dans une même chaîne. Pour le positionnement, cela ressemble à :

```
# <ch> P <pw> S <spd> ... # <ch> P <pw> S <spd> T <time> <cr>
```

On ne peut pas mélanger des commandes de types différents dans un même groupe. La fonction <time> doit être placée en fin de ligne et concerne l'ensemble du mouvement, tandis que <speed> peut être associée à chaque servo.

Exemple 1 :

```
#5 P1600 #10 P750 T2500 <cr>
```

Cet exemple déplace le servo 5 en position 1600 et le servo 10 en position 750 ; le mouvement prend 2500 ms (2,5 s), même si un servo doit parcourir plus de chemin que l'autre. Les deux servos démarrent et s'arrêtent en même temps. C'est une commande très puissante : en commandant toutes les pattes d'un robot marcheur par un « Group Move », il est facile de synchroniser des démarches complexes. La même synchronisation profite aussi au pilotage d'un bras robotisé.

Vous pouvez combiner les commandes de vitesse et de temps. La vitesse de chaque servo est alors calculée selon les règles suivantes :

1. Tous les canaux démarrent et terminent le mouvement simultanément.
2. Si une vitesse est spécifiée pour un servo, il ne se déplacera pas plus vite que cette vitesse (mais il peut aller plus lentement si le temps l'exige).
3. Si un temps est spécifié pour le mouvement, celui-ci durera au moins ce temps (mais peut durer plus longtemps si la vitesse l'exige).

Exemple 2 :

```
#5 P1600 #17 P750 S500 #2 P2250 T2000 <cr>
```

Cet exemple applique 1600 µs sur le canal 5, 750 µs sur le canal 17 et 2250 µs sur le canal 2. Le mouvement complet prendra au moins 2 secondes, mais le canal 17 ne dépassera pas 500 µs/s. Le temps réel dépendra de la position initiale du canal 17 : s'il part de 2000, il doit parcourir 1250 µs ; limité à 500 µs/s, il lui faut au moins 2,5 s, donc tout le mouvement dure 2,5 s. S'il part de 1000, il ne parcourt que 250 µs (0,5 s) et le mouvement dure 2 s.

⚠ Important ! La première commande de positionnement doit être une commande normale « # <ch> P <pw> ». Comme le contrôleur ignore la position des servos à la mise sous tension, il ignorera les commandes de vitesse et de temps jusqu'à réception de la première commande normale.

Tout mouvement impliquant plusieurs servos et utilisant le modificateur S ou T est considéré comme un « Group Move » : tous les servos démarrent et s'arrêtent en même temps. Pour déplacer plusieurs servos à des vitesses différentes, il faut émettre les commandes séparément.

Décalage de position logiciel (Servo Position Offset)

```
#<ch> P0 <valeur_offset> ... #<ch> P0 <valeur_offset> ... <cr>
```

- **<ch>** : numéro de canal en décimal (0 à 31)
- **<offset de position>** : limité à -100 µs à +100 µs (environ 15°)
- **<cr>** : retour chariot

La commande de décalage de position permet de modifier par logiciel la position centrale (associée à 1500 µs) d'un ou plusieurs servos, dans une plage de 15° autour du centre absolu. Elle n'a besoin d'être

envoyée qu'une seule fois, au démarrage du programme. Attention : la SSC-32U oublie ces décalages lorsqu'elle est mise hors tension. Le servo lui-même n'est pas modifié ; seul le signal envoyé à la (aux) broche(s) concernée(s) est altéré.

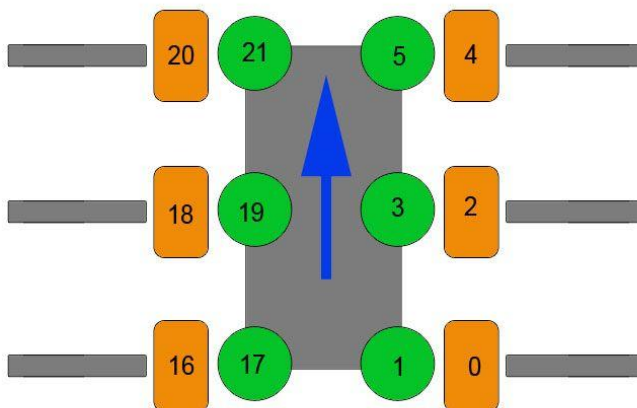
L'un des usages majeurs est de centrer / aligner parfaitement la position d'un servo avec la mécanique à laquelle il est relié. Par exemple, en assemblant un servo sur un système de support, vous pouvez constater un désalignement de quelques degrés alors que votre code exige un alignement parfait. Corriger cela mécaniquement étant très difficile, la commande de décalage permet un réglage fin. Il reste important de construire l'assemblage mécanique aussi proche que possible de la position centrée avant d'appliquer le décalage.

Séquenceur hexapode 12 servos

La SSC-32U intègre une fonction permettant de piloter facilement un robot marcheur hexapode à 12 degrés de liberté (six pattes, deux degrés de liberté par patte). Cette fonction nécessite toujours l'envoi de commandes série depuis une source externe (microcontrôleur, ordinateur ou module sans fil).

Les canaux servo utilisés par le séquenceur hexapode sont les suivants (vertical / horizontal se rapportent au mouvement) :

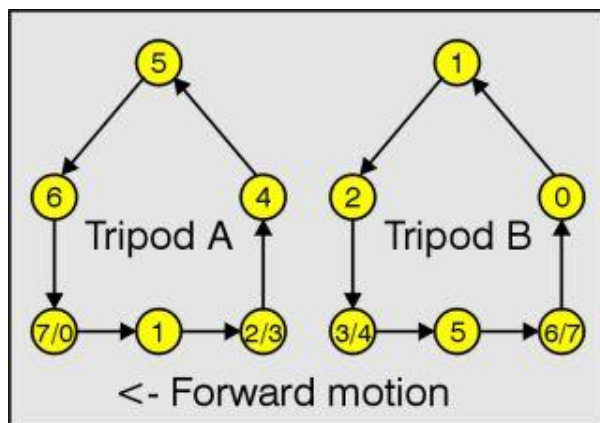
Canal	Fonction (côté droit)	Canal	Fonction (côté gauche)
0	Arrière droit — Vertical	16	Arrière gauche — Vertical
1	Arrière droit — Horizontal	17	Arrière gauche — Horizontal
2	Centre droit — Vertical	18	Centre gauche — Vertical
3	Centre droit — Horizontal	19	Centre gauche — Horizontal
4	Avant droit — Vertical	20	Avant gauche — Vertical
5	Avant droit — Horizontal	21	Avant gauche — Horizontal



Affectation des broches servo de l'hexapode

La séquence de marche comporte 8 états, numérotés de 0 à 7. La démarche utilise un trépied alterné (trois pattes levées, trois pattes au sol), chaque pied passant par six positions possibles. Les deux trépieds sont nommés Trépied A et Trépied B :

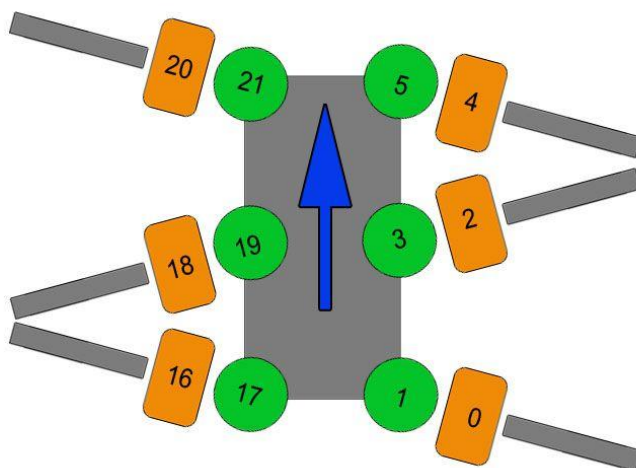
- Trépied A : {patte avant gauche, patte arrière gauche, patte centre droit}
- Trépied B : {patte centre gauche, patte avant droit, patte arrière droit}



Vue de côté des pattes pour chaque groupe de trois pattes

Pendant la marche, les pattes passent par 6 points : Avant bas (Low Front), Centre bas (Low Center), Arrière bas (Low Rear), Arrière milieu (Mid Rear), Centre haut (High Center) et Avant milieu (Mid Front). « Centre » désigne le point médian entre l'avant et l'arrière.

État	Servo vertical (Trépied A)	Servo horizontal (Trépied A)	Servo vertical (Trépied B)	Servo horizontal (Trépied B)
0	Bas	Avant → Centre	Milieu → Haut	Arrière → Centre
1	Bas	Centre → Arrière	Haut → Milieu	Centre → Avant
2	Bas	Arrière	Milieu → Bas	Avant
3	Bas → Milieu	Arrière	Bas	Avant
4	Milieu → Haut	Arrière → Centre	Bas	Avant → Centre
5	Haut → Milieu	Centre → Avant	Bas	Centre → Arrière
6	Milieu → Bas	Avant	Bas	Arrière
7	Bas	Avant	Bas → Milieu	Arrière



Position et mouvement du pied durant la marche

Configuration de la démarche

LH <arg>, LM <arg>, LL <arg>

Règlent la valeur des servos verticaux du côté gauche de l'hexapode. LH = valeur haute (largeur d'impulsion pour lever la patte au maximum) ; LM = valeur milieu ; LL = valeur basse. Plage valide des arguments : 500 à 2500 μ s.

RH <arg>, RM <arg>, RL <arg>

Règlent la valeur des servos verticaux du côté droit. RH = valeur haute ; RM = valeur milieu ; RL = valeur basse. Plage valide : 500 à 2500 μ s.

VS <arg>

Règle la vitesse de déplacement des servos verticaux. Tous les mouvements verticaux utilisent cette vitesse. Plage valide : 0 à 65535 μ s/s.

LF <arg>, LR <arg>

Règlent la valeur des servos horizontaux du côté gauche. LF = valeur avant (largeur d'impulsion pour la position la plus avancée) ; LR = valeur arrière. Plage valide : 500 à 2500 μ s.

RF <arg>, RR <arg>

Règlent la valeur des servos horizontaux du côté droit. RF = valeur avant ; RR = valeur arrière. Plage valide : 500 à 2500 μ s.

HT <arg>

Règle le temps de déplacement entre les positions horizontales avant et arrière. Plage valide : 1 à 65535 μ s.

XL <arg>, XR <arg>

Règlent le pourcentage de course des pattes gauches et droites. Plage valide : -100 % à 100 %. Les valeurs négatives font reculer les pattes du côté concerné. À 100 %, les pattes se déplacent entre les positions avant et arrière. Des valeurs plus faibles réduisent proportionnellement la course, toujours centrée. La vitesse des mouvements horizontaux est ajustée en fonction de XL et XR, de sorte que le temps de déplacement reste constant.

XS <arg>

Règle le pourcentage de vitesse horizontale de toutes les pattes. Plage valide : 0 % à 200 %. À 100 %, le temps de course horizontale est la valeur programmée par HT. Des valeurs plus élevées réduisent proportionnellement le temps de course ; des valeurs plus faibles l'augmentent. À 0 %, le robot s'arrête sur place. Le séquenceur hexapode ne démarre qu'après réception de la commande XS.

XSTOP

Arrête le séquenceur hexapode et ramène tous les servos en fonctionnement normal.

1. Lorsqu'un servo horizontal se déplace, sa vitesse est ajustée selon les largeurs d'impulsion avant/arrière, le pourcentage XL/XR et le pourcentage XS. Quelle que soit la distance de course (ajustée par XL/XR), le temps total du mouvement est HT divisé par le pourcentage XS.
2. Lorsqu'un servo vertical passe de Bas à Milieu ou de Milieu à Bas, il se déplace à la vitesse VS. Lorsqu'il passe de Milieu à Haut ou de Haut à Milieu, sa vitesse verticale est ajustée pour que les mouvements horizontal et vertical se terminent en même temps.
3. N'importe quelle commande du séquenceur peut être émise pendant son fonctionnement : elle prend effet immédiatement.

Exemples pour le séquenceur hexapode :

```
"LH1000 LM1400 LL1800 RH2000 RM1600 RL1200 VS3000 <cr>" → règle les paramètres des servos verticaux.
"LF1700 LR1300 RF1300 RR1700 HT1500 <cr>" → règle les paramètres des servos horizontaux.
"XL50 XR100 XS100 <cr>" → virage progressif à gauche à 100 % (et démarre le séquenceur s'il ne l'est pas).
"XL -100 XR 100 XS 50 <cr>" → rotation sur place vers la gauche à 50 %.
"XSTOP <cr>" → arrête le séquenceur et redonne le contrôle normal aux canaux 0-5 et 16-21.
```

Interroger l'état du séquenceur — XQ <cr>

Renvoie un chiffre représentant l'état du séquenceur, ainsi que le pourcentage approximatif d'avancement dans cet état. Le quartet de poids fort vaut « 0 » à « 7 », celui de poids faible « 0 » à « 9 ». Par exemple, si le séquenceur est à 80 % de l'état 5, il renvoie la valeur 58 (hex).

Fonctions avancées

Annuler une sortie (Cancel Output)

```
# <ch> P <pw> .... <esc>
```

<esc> : annule l'action en cours (ASCII 27). Pour annuler une commande, ajoutez un <esc> en fin de ligne.

Sortie tout-ou-rien (Discrete Output)

Les broches d'E/S de la SSC-32 peuvent fournir un niveau HAUT (5 V) ou BAS (0 V). N'utilisez PAS cette fonction avec des servos R/C standard.

```
# <ch> <lvl> ... # <ch> <lvl> <cr>
```

- **<ch>** : numéro de canal en décimal, 0-31
- **<lvl>** : niveau logique du canal, « H » pour Haut ou « L » pour Bas
- **<cr>** : retour chariot (ASCII 13)

Les sorties de la SSC-32 proviennent de quatre registres à décalage 8 bits. Il y a quatre bancs de 8 sorties : 0-7, 8-15, 16-23 et 24-31. Chaque broche peut absorber ou fournir jusqu'à 20 mA, mais un maximum de 70 mA par banc doit être respecté. Le canal passe au niveau indiqué dans les 20 ms suivant la réception du retour chariot.

Exemple :

```
#3H #4L <cr>
```

Met un niveau Haut (+5 V) sur le canal 3 et un niveau Bas (0 V) sur le canal 4.

Sortie par octet (Byte Output)

```
# <bank> : <value> <cr>
```

- **<bank>** : 0 = broches 0-7, 1 = broches 8-15, 2 = broches 16-23, 3 = broches 24-31
- **<value>** : valeur décimale à écrire sur le banc (0-255), le bit 0 étant le bit de poids faible
- **<cr>** : retour chariot (ASCII 13)

Cette commande écrit 8 bits de données binaires d'un coup. Toutes les broches du banc sont mises à jour simultanément, dans les 20 ms suivant le retour chariot. Le deux-points (:) est obligatoire.

Exemple :

```
#3:123 <cr>
```

Écrit la valeur 123 (décimal) sur le banc 3. 123 = 01111011 (binaire) ; le banc 3 correspond aux broches 24-31. Cette commande met donc un « 0 » sur les broches 26 et 31, et un « 1 » sur toutes les autres.

Interroger l'état du mouvement — Q <cr>

Renvoie un « . » si le mouvement précédent est terminé, ou un « + » s'il est encore en cours. Un délai de 50 µs à 5 ms précède la réponse.

Interroger la largeur d'impulsion — QP <arg> <cr>

Renvoie un octet (au format binaire) indiquant la largeur d'impulsion du servo sélectionné, avec une résolution de 10 μ s. Par exemple, pour une largeur de 1500 μ s, l'octet renvoyé sera 150. Plusieurs servos peuvent être interrogés dans la même commande : la réponse comporte un octet par servo. Un délai d'au moins 50 μ s à 5 ms précède la réponse (généralement < 100 μ s).

Entrée numérique (Digital Input)

```
A B C D E F AL BL CL DL EL FL <cr>
```

A, B, C, D, E, F sont des lectures d'entrée normales : elles lisent la valeur binaire de l'entrée. Renvoie « 0 » (ASCII) si l'entrée est basse (0 V) ou « 1 » si elle est haute (+5 V). Les broches G et H ne sont pas incluses (elles sont uniquement analogiques).

AL, BL, CL, DL, EL, FL sont des lectures à verrouillage (latch) : elles renvoient « 0 » si l'entrée est basse ou si elle a été basse depuis la dernière commande *L. Elles renvoient « 1 » si l'entrée est haute et n'est jamais passée à l'état bas depuis la dernière commande *L. En clair, elles renvoient un niveau bas si l'entrée est passée bas à un moment quelconque. Lire l'état réinitialise le verrou.

Ces entrées disposent d'une faible résistance de tirage (pull-up, ~50 k Ω) activée en mode entrée. Elles sont scrutées environ toutes les 1 ms et anti-rebond pendant ~15 ms. La valeur logique d'une lecture ne change qu'après 15 ms de stabilité au nouveau niveau. Les lectures peuvent être groupées (jusqu'à 8 valeurs) ; la réponse est une chaîne d'un caractère par entrée, sans espaces.

Exemple :

```
A B E DL <cr>
```

Entrées analogiques (Analog Inputs)

```
VA VB VC VD VE VF VG VH <cr>
```

Les broches A à H sont des broches d'entrée/sortie analogiques, utilisables pour lire des capteurs, piloter des LED de faible puissance, etc. VA, VB, VC, VD lisent la valeur analogique de l'entrée : renvoie un octet avec la valeur binaire 8 bits de la tension sur la broche.

Lorsque les entrées A, B, C, D servent d'entrées analogiques, le pull-up interne est désactivé. Les entrées sont filtrées numériquement pour réduire le bruit ; les valeurs se stabilisent dans les 8 ms suivant un changement. La valeur 0 représente 0 V, la valeur 255 représente +4,98 V. Pour convertir en tension, multipliez par 5/256. À la mise sous tension, A-B-C-D sont configurées en entrée numérique avec pull-up ; à la première commande V*, la broche passe en analogique sans pull-up et le résultat de cette première lecture n'est pas valide.

Exemple :

```
VA VB <cr>
```

Renvoie 2 octets avec les valeurs analogiques de A et B. Par exemple, si la tension sur A est 2 V et sur B 3,5 V, les octets renvoyés seront 102 et 179.

Débit (Baud)

La SSC-32U est livrée avec un débit par défaut de 9600 bauds. Le débit peut aussi être réglé avec le bouton-poussoir embarqué :

1. Maintenez le bouton enfoncé. Les LED s'allument pour indiquer le débit actuel :
 - 9600 (vert)
 - 38400 (rouge)
 - 115200 (vert et rouge)
 - Débit non standard (aucune LED)
2. Après 2 secondes, les LED se mettent à alterner : vous pouvez changer le débit.
3. Relâchez le bouton.
4. Appuyez sur le bouton pour faire défiler les débits de l'étape 1.
5. Une fois le débit choisi, ne faites rien : après 5 secondes, les LED reviennent en mode normal et le nouveau débit est écrit en EEPROM.

Le registre R4 contient désormais le débit. Il peut aussi être écrit par ordinateur. Pour faire tenir les débits dans une valeur 16 bits, la valeur est stockée divisée par 10 : un débit de 9600 bauds est donc stocké comme 960. Par exemple, pour régler 2400 bauds, envoyez R4=240. Dans ce cas, appuyer sur le bouton n'allume pas les LED (débit non standard), mais les fait clignoter brièvement. Vous pouvez relire le débit avec la commande « R4 » suivie d'un retour chariot ; « R4<cr> » renverra « 240 » si le débit est 2400.

Firmware et registres

Pour mettre à jour le firmware, utilisez l'utilitaire gratuit SSC-32 Servo Sequencer Utility (RB_Dsp-07).

Registres de la SSC-32 — informations générales

N°	Nom	Minimum	Maximum	Défaut	Description
0	Enable	0	65535	0	Champ de bits (16 bits) activant diverses fonctions de la SSC-32.
1	Transmit Delay	0	65535	600	Délai (μ s) avant l'émission du premier octet d'une réponse.
2	Transmit Pacing	0	65535	70	Délai (μ s) entre les octets d'une réponse.
3-31	(Réservés)	—	—	—	—
32-63	Initial Pulse Offset	-100	100	0	Valeur initiale du décalage d'impulsion (PO) pour chaque servo. R32 = servo 0, R33 = servo 1, etc.
64-95	Initial Pulse Width	0	65535	1500	Largeur d'impulsion initiale pour chaque servo. R64 = servo 0, etc. 0 = sortie logique « 0 » continue ; 65535 = logique « 1 » continue ; les autres valeurs sont bornées à 500-2500 μ s.
96-255	(Réservés)	—	—	—	—

⚠ Les registres 0-15 sont à usage global (affectent tout le fonctionnement de la SSC-32) ; les registres 32-255 servent à la configuration individuelle de chaque canal servo, par groupes de 32 registres.

Registre Enable (R0) — définition des bits

Bit	Nom	Définition
15 (msb)	Global Disable	Si « 1 », désactive toutes les fonctions contrôlées par le registre Enable. Si « 0 », les bits individuels servent à activer/désactiver les fonctions.
14-4	(Réservés)	—
3	Initial Pulse Width Enable	Si « 1 », active les valeurs du registre Initial Pulse Width au démarrage. Si « 0 », la valeur par défaut 0 est utilisée.
2	Initial Pulse Offset Enable	Si « 1 », active les valeurs du registre Initial Pulse Offset au démarrage. Si « 0 », la valeur par défaut 0 est utilisée.
1	TX Delay/Pacing Enable	Si « 1 », active les valeurs Transmit Delay et Transmit Pacing. Si « 0 », les valeurs par défaut (600 μ s et 70 μ s) sont utilisées.

Bit	Nom	Définition
0 (lsb)	Startup String Enable	Si « 1 », exécute la chaîne de démarrage à la mise sous tension. Si « 0 », elle n'est pas exécutée.

Lecture / écriture de registre

Commande	Argument	Description / Exemples
Écriture : R <r> = <n> <cr>	r = n° de registre (0-255), n = valeur	Programme la valeur d'un registre. Les espaces sont facultatifs. Ex. : R0=1023 <cr> · R32 = -50 <cr>
Lecture : R <r> <cr>	r = n° de registre (0-255)	Affiche la valeur du registre, suivie d'un retour chariot (format ASCII). Ex. : R0 <cr> → 1023<cr>
Valeurs par défaut : RDFLT <cr>	aucun	Remet tous les registres à leur valeur par défaut. Une fois terminé, la SSC-32 émet « OK<cr> ».

⚠ La commande RDFLT peut prendre plus d'une seconde. Ne l'invoquez pas pendant un mouvement temporisé ou une séquence active, et n'écrivez aucun registre avant la fin (réponse « OK »). Si vous envoyez plusieurs commandes R= par logiciel, relisez chaque registre après écriture pour garantir la fin de l'opération avant la suivante.

⚠ N'éteignez pas la SSC-32 pendant l'exécution d'une commande RDFLT ou R=. Chaque écriture use légèrement l'EEPROM : le nombre maximal typique d'écritures est de 100 000. Ne programmez pas de changements rapides et répétés des registres, sous peine d'user prématurément l'EEPROM du processeur.

Commandes de registre diverses

Commande	Argument	Description
STOP <n> <cr>	0-31	Arrête immédiatement le servo indiqué à sa position actuelle. Un espace est facultatif avant le numéro. Ex. : STOP0 <cr> · STOP 31 <cr>

Si le servo fait partie d'un mouvement temporisé, les autres servos continuent et une commande d'interrogation indiquera que le mouvement se poursuit jusqu'à expiration du temps total initial — même si tous les servos du mouvement initial sont arrêtés.

⚠ Les commandes EER et EEW ne fonctionnent plus pour accéder à l'EEPROM interne ; elles sont remplacées par les commandes de lecture/écriture de registre et de chaîne de démarrage. EER et EEW continuent de fonctionner pour l'EEPROM externe.

Chaînes de démarrage (Startup Strings)

Commande	Argument	Description / Exemples
Supprimer : SSDEL <n> <cr>	0-255	Supprime <n> caractères à la fin de la chaîne de démarrage. Si <n> dépasse la longueur, supprime toute la chaîne. Ex. : SSDEL 5 <cr> · SSDEL 255 <cr>
Concaténer : SSCAT <chaîne> <cr>	jusqu'à 100 caractères ASCII	Ajoute <chaîne> à la chaîne de démarrage actuelle. L'espace immédiatement après « SSCAT » est ignoré ; les autres espaces font partie de la chaîne. Terminée par un retour chariot, sans retour chariot intégré. Les commandes sont séparées par un point-virgule (y compris la dernière). Ex. : SSCAT #0P1000#1P2000T3000;<cr>
Afficher : SS <cr>	aucun	Affiche toute la chaîne de démarrage, entre guillemets, suivie d'un retour chariot.

La chaîne de démarrage programmée est exécutée à la mise sous tension de la SSC-32, si le bit « Startup String Enable » du registre Enable est positionné. Elle est exécutée après l'application des valeurs de registre (ex. largeur d'impulsion initiale). Sa longueur totale maximale est de 100 caractères ASCII ; les caractères supplémentaires sont ignorés.

⚠ Les commandes suivantes ne doivent pas figurer dans une chaîne de démarrage : EER, EEW, R=, SSCAT, SSDEL. Les commandes SS et SSCAT peuvent prendre plusieurs centaines de millisecondes ; ne les invoquez pas pendant un mouvement temporisé ou une séquence active. N'éteignez pas la carte pendant l'exécution de SSDEL ou SSCAT (vérifiez la fin avec SS). L'EEPROM s'use à chaque modification (max ~100 000 écritures).

Exemples de chaînes de démarrage

Commande	Résultat
SSDEL 255 <cr> puis SS <cr>	""<cr>
SSCAT #0P2000T5000;<cr> puis SS <cr>	"#0P2000T5000;"<cr>
SSCAT XXXX<cr> puis SS <cr>	"#0P2000T5000;XXXX"<cr>
SSDEL 4 <cr> puis SS <cr>	"#0P2000T5000;"<cr>
SSDEL 6 <cr> puis SS <cr>	"#0P2000"<cr>

Exemples supplémentaires

Commande	Résultat
RDFLT	Remet tous les registres aux valeurs par défaut
SSDEL 255	Efface la chaîne de démarrage
R0	Affiche le registre 0
R0=2 / R1=2000 / R2=1000	Règle le délai TX à 2000 µs et le pacing TX à 1000 µs (bit 1 de R0 active TX delay/pacing)
R0=12 / R32=50 / R64=1000	Décalage du servo 0 à 50 et largeur initiale à 1000 (bits 2 et 3 de R0)

Commande	Résultat
R0=13 / SSDEL 255 / SSCAT #0P1500T5000;	Déplace lentement le servo 0 à 1500 au démarrage (bit 0 de R0 active la chaîne de démarrage)